

Construire un Agent IA de Recherche Approfondie avec LangGraph

Poser la Recherche,
Analyser, Itérer,
Obtenir de Meilleures
Réponses !

① Qu'est-ce que nous construisons ?

- Un Agent IA de Recherche Approfondie qui peut :
 - Comprendre la question de recherche
 - Créer un plan de recherche
 - Générer des requêtes de recherche
 - Rechercher sur le web (Tavily)
 - Analyser les résultats
 - Décider si plus de recherche est nécessaire
 - Générer la réponse finale

Au lieu d'un seul appel LLM, nous construisons un flux de travail de recherche complet.

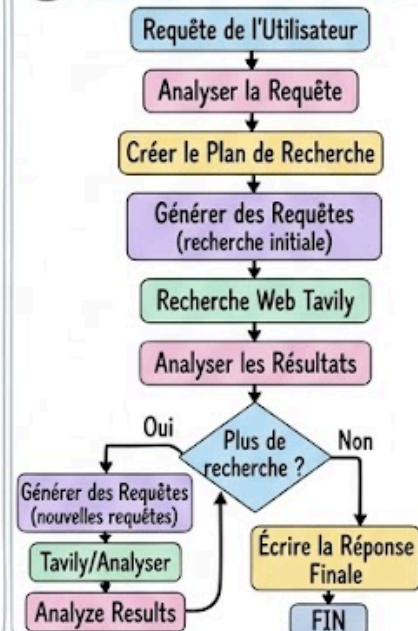
② Exemple de Question

« Quels frameworks d'agents IA sont les meilleurs pour les applications de production en 2026 ? »

Things à rechercher :

- Préparation à la production
- Gestion d'état et persévérance
- Appel d'outils et orchestration
- Support multi-agent
- Observabilité et débogage
- Déploiement et scalabilité
- Expérience développeur et écosystème

③ Architecture (Flux LangGraph)



④ Créer un Environnement Virtuel

1. Créer un dossier de projet

```
mkdir deep-research-agent
cd deep-research-agent
```

2. Créer un environnement virtuel

```
python -m venv .venv
```

3. L'activer

macOS / Linux:

```
source .venv/bin/activate
```

Windows:

```
.venv\Scripts\activate
```

- ✓ Vous devriez voir (.venv) dans votre terminal.

⑤ Installer les Dépendances

Nous avons besoin de :

- LangGraph
- L'intégration OpenAI de LangChain
- python-dotenv
- Client Python Tavily

Installer en utilisant :

```
python -m pip install \
langgraph langchain-openai \
python-dotenv tavily-python
```

⑥ Configurer les Clés API

1. Créer un fichier .env
2. Ajouter vos clés API :



```
OPENAI_API_KEY=votre_cle_api_openai
TAVILY_API_KEY=votre_cle_api_tavily
```

- Les charger en utilisant `load_dotenv()`
- Ne committez pas votre fichier .env sur GitHub.
- Ajoutez .env à votre fichier .gitignore.

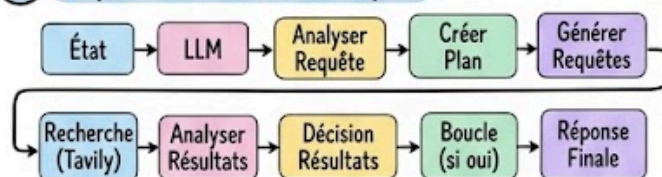
⚠ Gardez vos clés API privées !

⑦ Structure du Projet

```
+ deep-research-agent/
  .venv/
  .env
  .gitignore
  state.py
  nodes.py
  graph.py
  main.py
```

Chaque fichier a un rôle spécifique dans notre agent LangGraph.

⑧ Étapes Suivantes (Flux Complet)



Cela nous donne un flux de travail complet de Recherche Approfondie !

Guide Complet pour Débutant : Construire un Agent IA de Recherche Approfondie avec LangGraph

Ce cours pratique et progressif s'adresse à toute personne souhaitant comprendre et concevoir un agent d'intelligence artificielle autonome capable de mener des investigations approfondies sur le web. Structuré en huit étapes fondamentales, il détaille le passage d'une simple requête textuelle à une véritable architecture logicielle en graphe.

1. Qu'est-ce qu'un Agent de Recherche Approfondie (Deep Research) ?

Pour un utilisateur débutant, la première étape consiste à distinguer un appel standard à un Modèle de Langage (LLM) comme ChatGPT d'un véritable système agentique.

- **L'approche classique (requête unique)** : Vous posez une question, le modèle puise dans sa mémoire statique (ou effectue une recherche web superficielle) et génère une réponse immédiate. Si les informations sont insuffisantes ou obsolètes, il risque d'halluciner.
- **L'approche agentique (workflow itératif)** : L'agent adopte la démarche d'un analyste humain. Il commence par formuler un plan, lance des requêtes sur les moteurs de recherche spécialisés, lit et analyse les pages web retournées, évalue si des données manquent, et itère jusqu'à obtenir un niveau de preuve suffisant avant de rédiger sa synthèse.

Critère	Appel LLM Standard	Agent Deep Research
Méthode d'exécution	Linéaire (entrée → sortie directe)	Cyclique avec boucle de rétroaction
Recherche d'informations	Nulle ou limitée à 1 seule requête	Multiples requêtes ciblées via API spécialisée
Autonomie & Vérification	Aucune vérification critique interne	Auto-évaluation des lacunes et relance automatique

2. Définition de la Problématique et Décomposition

Un bon agent ne lance jamais des recherches au hasard. Il applique une méthode d'ingénierie cognitive : analyser une question complexe pour la décomposer en sous-objectifs clairs et vérifiables.

Exemple pratique : « *Quels frameworks d'agents IA sont les plus adaptés aux applications de production en 2026 ?* »

Pour traiter cette demande sans survoler le sujet, l'agent identifie automatiquement les angles thématiques indispensables :

1. **Maturité en production (Production readiness)** : Robustesse, gestion des erreurs, garanties de disponibilité.
2. **Gestion d'état et persistance** : Capacité à sauvegarder les sessions et reprendre l'exécution après une interruption.
3. **Appel d'outils et orchestration** : Fiabilité du function calling et contrôle des flux d'exécution.
4. **Support multi-agents** : Coordination et communication structurée entre agents spécialisés.
5. **Observabilité et traçabilité** : Logs, débogage et suivi des coûts de tokens.
6. **Déploiement et passage à l'échelle** : Conteneurisation, gestion de la charge et latence.
7. **Expérience développeur et écosystème** : Clarté de la documentation et vitalité de la communauté.

3. Comprendre l'Architecture en Graphe avec LangGraph

LangGraph est une bibliothèque développée par LangChain conçue pour créer des applications agentiques cycliques et avec état (stateful). Contrairement aux chaînes traditionnelles purement séquentielles (DAG), LangGraph permet d'implémenter des boucles logiques.

Le flux décisionnel fonctionne selon les étapes suivantes :

1. **Requête de l'utilisateur (User Query)** : Point de départ du processus.
2. **Analyse de la requête (Analyze Request)** : Le LLM extrait les intentions directrices et les contraintes.
3. **Création du plan de recherche (Create Research Plan)** : Établissement des étapes ordonnées d'investigation.
4. **Génération des requêtes web (Generate Search Queries)** : Production de mots-clés spécifiques adaptés aux moteurs de recherche.
5. **Recherche Web Tavily (Tavily Web Search)** : Récupération des contenus nettoyés et pertinents via l'API.
6. **Analyse des résultats (Analyze Results)** : Extraction des faits clés et mise en mémoire

dans l'état global.

7. **Nœud conditionnel de décision (Need more research?) :**
 - *Si Oui* : Le graphe reboucle pour générer de nouvelles requêtes ciblées comblant les zones d'ombre.
 - *Si Non* : Le graphe oriente le flux vers l'étape finale.
8. **Rédaction de la réponse finale (Write Final Answer) :** Synthèse structurée, documentée et sourcée.
9. **Fin (END) :** Restitution du résultat à l'utilisateur.

4. Mise en Place de l'Environnement de Développement

Pour éviter tout conflit entre les versions de packages installés sur votre système, il est indispensable de créer un environnement virtuel Python dédié à votre projet.

Ouvrez votre terminal et exécutez les commandes suivantes :

```
# 1. Créer le dossier du projet et s'y déplacer
mkdir deep-research-agent
cd deep-research-agent
```

```
# 2. Créer l'environnement virtuel Python nommé .venv
python -m venv .venv
```

```
# 3. Activer l'environnement virtuel
# Sur macOS ou Linux :
source .venv/bin/activate
```

```
# Sur Windows (Invite de commandes ou PowerShell) :
.venv\Scripts\activate
```

Une fois activé, l'indicateur (.venv) apparaît en préfixe de votre ligne de commande dans le terminal, confirmant que vos futures installations seront confinées à ce projet.

5. Installation et Rôle des Dépendances Logicielles

Quatre bibliothèques principales composent la pile technique de notre agent :

- **langgraph** : Le moteur d'orchestration orchestrant les états, les nœuds et les transitions cycliques.
- **langchain-openai** : Le pont d'intégration permettant de communiquer avec les modèles

de fondation d'OpenAI (ex. GPT-4o).

- **python-dotenv** : Outil de chargement automatique des variables d'environnement depuis un fichier local non versionné.
- **tavily-python** : Client officiel du moteur de recherche Tavily, spécialement optimisé pour les agents IA (retourne du texte pré-nettoyé sans publicités ni balises superflues).

Installez l'ensemble des bibliothèques en une seule ligne de commande :

```
python -m pip install langgraph langchain-openai python-dotenv tavily-python
```

6. Configuration Sécurisée des Clés d'API

Les agents interagissent avec des services externes sécurisés par jetons. Ces jetons sont confidentiels et ne doivent jamais figurer en clair dans votre code source ou sur un dépôt public tel que GitHub.

1. Créez un fichier nommé `.env` à la racine du projet.
2. Définissez vos clés sous forme de paires clé-valeur :

```
OPENAI_API_KEY=votre_cle_api_openai_secrete  
TAVILY_API_KEY=votre_cle_api_tavily_secrete
```

Créez immédiatement un fichier `.gitignore` à côté de votre fichier `.env` et ajoutez-y la ligne suivante :

```
.env  
.venv/  
__pycache__/
```

Dans votre code Python, la fonction `load_dotenv()` de la bibliothèque `dotenv` injectera automatiquement ces valeurs dans l'environnement d'exécution de votre script en toute sécurité.

7. Organisation et Structure Modulaire du Code

Une application professionnelle évite le code monolithique. Le projet est découpé en modules spécialisés selon une responsabilité unique :

Fichier	Rôle architectural	Description pour débutant
state.py	Schéma de données (State)	Définit le tableau de bord partagé (mémoire) qui transite de nœud en nœud au fil de l'exécution.
nodes.py	Nœuds d'action (Workers)	Contient les fonctions Python exécutées à chaque étape (appel au LLM, interrogation de Tavily, analyse).
graph.py	Réseau et conditions (Wiring)	Relie les nœuds entre eux, configure les arêtes standard et les arêtes conditionnelles (la boucle de retour).
main.py	Point d'entrée (Runtime)	Déclenche l'exécution en injectant la question initiale de l'utilisateur et affiche le rapport généré.

8. Exécution du Cycle Complet et Synthèse Finale

L'exécution de l'agent suit une séquence d'états immuables et enrichis :

1. **Initialisation de l'état** : La structure AgentState reçoit la question de départ et initialise une liste vide pour les résultats de recherche et un compteur d'itérations.
2. **Génération du plan et des requêtes** : Le modèle produit des requêtes précises. Par exemple, au lieu de chercher "framework IA", il cherchera : *"LangGraph production latency benchmarks"* ou *"LangGraph vs Autogen state persistence comparison"*.
3. **Collecte Tavily** : L'outil extrait le texte brut, synthétise les métadonnées et alimente la mémoire d'état.
4. **Évaluation critique** : Le nœud d'analyse vérifie si chaque axe du plan initial possède des données probantes. Si une question demeure sans réponse (par exemple le déploiement multi-cloud), le nœud conditionnel ordonne une nouvelle itération en posant une question plus précise.
5. **Synthèse et restitution finale** : Dès que l'ensemble des axes est documenté ou que la

limite d'itérations définie (ex. 3 boucles maximum) est atteinte, le modèle de rédaction prend l'ensemble des notes accumulées dans l'état et compile un rapport structuré avec références et recommandations claires.

Résumé des Clés de Réussite

Pour bâtir un agent fiable en tant que débutant :

- Contrôlez toujours la boucle d'itération par une condition de sortie stricte (seuil de qualité ou nombre maximum de boucles) pour éviter les dépenses d'API incontrôlées.
- Conservez l'état (State) le plus épuré possible en ne stockant que les résumés pertinents plutôt que des pages web entières.
- Privilégiez la modularité du code pour faciliter le débogage de chaque nœud séparément.